

an application called 'cms' for our project, and it will appear as a directory called 'cms' within our project folder along with the 'mycms' directory and the 'manage.py' file.

Let's examine the structure of this directory:

Name	Size	Date
▼ migrations	1 item	2015-07-15 15:31
__init__.py	0 B	2015-07-15 15:31
__init__.py	0 B	2015-07-15 15:31
admin.py	63 B	2015-07-15 15:31
models.py	57 B	2015-07-15 15:31
tests.py	60 B	2015-07-15 15:31
views.py	63 B	2015-07-15 15:31

django-newapp

admin.py: The code that sets up the administrative interface for our website

models.py: Database models for our website

tests.py: Here you can write unit tests for your application

views.py: The views contain code that renders the actual content that will appear in a user's browser

__init__.py: As mentioned before, this makes the 'cms' directory a Python package, ready for importing

migrations: As you make changes to the structure of your database, Django keeps track of the changes made in this folder so you can go from an older structure to a newer structure without losing data.

Now that we have our 'cms' application set up, we need to include this application in our project by adding it to the appropriate place in the 'settings.py' file. Open the 'settings.py' file in the 'mycms' directory and look for the 'INSTALLED_APPS' setting that contains a tuple of installed applications. By default it will include some of the applications that are distributed with Django. We need to add our newly created 'cms' application to this list.

Whenever you modify your application in a way that affects the database, you need to do two things:

First, you need to run the 'python3 manage.py makemigrations' com-